

Dublin Business School



B9AI002 - CA02

MSc AI Group A

Module Name: B9AI002

Lecturer Name: Prof. Terri Hoare

Student Name: Dilani Nadeesha Jayasuriya

Student ID: 20093736

**Group Members: Dilani Nadeesha (20093706), Mugdha Rajesh
(20088706)**

Programme: Master of Science in Artificial Intelligence

Submission Date: 19 April 2026

Contents

Introduction.....	4
Business Understanding.....	4
Data Understanding	5
Main Data Fields.....	6
Why graph structure is suitable.....	6
Data Preparation.....	7
Knowledge acquisition and transformation	7
Normalization	7
Graph-ready files	8
Neo4j loading.....	8
Validation	8
Exploratory Statistical and Visual Analysis	10
Most versatile plants	10
Dominant treatment categories	11
Access pattern analysis	12
Safety analysis	13
Knowledge Graph Design and Visualization	14
Modelling: Graph Algorithms and Graph Learning.....	15
Degree Centrality	15
Betweenness Centrality.....	16
Louvain Community Detection.....	17
Node Similarity	18
KNN on graph embeddings	19
Machine Learning Extension	20
Feature engineering.....	20
K-means clustering	21
Lasso regression.....	23
Interpretation.....	24
Evaluation	24
Evaluation of graph algorithms.....	24
Evaluation of graph ML.....	25
Evaluation of K-means	25

Evaluation of Lasso.....	25
Limitations	26
Deployment.....	27
Condition-to-remedy assistant	27
Safety-first version.....	28
Explainability.....	28
Future deployment	29
Conclusion	29
References.....	30

Introduction

This report presents the finding of a graph analytics and AI case study based on Irish medicinal plant and their usage in treating diseases (OCHTRINIL'S LEGACY). The study was conducted using 51 Irish medicinal plants, considering their associated ailments, categories, locations and mapped shops in Dublin city where plant-based medicines are available. These attributes were loaded into a Neo4j knowledge graph, and applied graph learning techniques were applied to generate insights and, while machine learning algorithms were used for deeper analysis. This framework followed stages of business understanding, data understanding, data preparation, modeling, evaluation, and deployment.

And Traditional medicinal knowledge is difficult to represent in a two-dimensional format because this domain contains many to many relationships. A single plant may treat multiple ailments, one ailment may be associated with many plants, and a plant may be linked to multiple wild locations or modern retail sources. A graph-based approach is appropriate because it captures the relational structure of the domain and supports both explainable querying and graph-based machine learning.

Business Understanding

The domain use case is the exploration of Irish medicinal plant knowledge. Traditional ethnobotanical knowledge is often fragmented across historical texts, folk practices, geographical references. This makes discovery, comparison, and recommendation difficult.

The main business problem addressed in this project is: **how can fragmented medicinal plant knowledge be transformed into an intelligent and explainable graph-based system that supports remedy discovery, similarity analysis, and safe condition-based search?**

The objectives of the project were:

- To build a Neo4j knowledge graph representing Irish medicinal plants and their relationships
- To identify highly connected and versatile plants
- To explore therapeutic groupings and structural importance using graph algorithms
- Comparing traditional wild access and modern retail access
- To build recommendation-style graph AI features using similarity
- To extend the work with graph-derived machine learning features.
- To propose an operational AI deployment scenario

Data Understanding

The dataset was custom-built from Irish medicinal plant information and structured into a graph-ready format. The final graph included the following entity types:

- Plant
- Ailment
- Ailment Category
- Family
- Wild Location
- Shop

The graph contained:

- 51 plant nodes
- 42 ailment nodes
- 34 family nodes
- 26 wild location nodes
- 22 shop nodes
- 13 ailment category nodes

The relationship structure contained:

- TREATS
- AVAILABLE_AT
- BELONGS_TO
- FOUND_IN
- IN_CATEGORY

These counts confirmed that the graph was sufficiently rich for both graph-theoretic and graph-learning analysis.

Main Data Fields

The source data included:

- Plant name
- Scientific name
- Irish name
- Plant family
- Toxicity label/ class
- Famine-food flag
- Cultural importance
- Medicinal uses
- Wild locations
- Dublin retail sources

Why is graph structure suitable

The medicinal plant domain is naturally relational. A graph data model allows connections between entities to be retained and explored directly. This is more suitable than a relational table because the analytical focus is not only on entity attributes but on the **structure of connections** between plants, ailments, categories, and access sources.



The screenshot shows a Neo4j Cypher query interface. The query entered is `neo4j$ // ===== // RESULT 1: NODE COUNTS`. Below the query, there are two tabs: 'Table' (selected) and 'RAW'. The results are displayed in a table with two columns: 'label' and 'count'. The table contains six rows of data, each with a line number in the first column.

	label	count
1	"Plant"	51
2	"Ailment"	42
3	"Family"	34
4	"WildLocation"	26
5	"Shop"	22
6	"AilmentCategory"	13

Figure 1: Node counts

neo4j\$ // ===== // RESULT 2: RELATIONSHIP COUNTS ,	
Table RAW	
relationship	count
1 "TREATS"	110
2 "AVAILABLE_AT"	53
3 "BELONGS_TO"	51
4 "FOUND_IN"	51
5 "IN_CATEGORY"	42

Figure 2 : Relationship counts

Data Preparation

Data preparation involved transforming semi-structured plant knowledge into graph-ready CSV files for Neo4j.

Knowledge acquisition and transformation

The dataset was not simply taken as a ready-made graph. Instead, the project involved a knowledge engineering stage where medicinal plant information was extracted from a research paper based on Irish women's knowledge of medicinal plants(Dolan, J.M., 2007), cleaned, normalized, and transformed into graph entities and relationships. This was one of the most important strengths of the project, because it's beyond downloading an existing dataset and instead involved creating a structured graph from domain knowledge from limited resources.

Normalization

Several normalization steps were applied:

- Ailment names were standardized
- Therapeutic categories were mapped to broader labels such as Respiratory, Skin/Wound, Digestive, and General Tonic
- Toxicity levels were standardized into low/ medium/ high style classes
- Retail source types were normalized into Retail Fresh, Retail Dry, Wild Only, and Mixed categories

Graph-ready files

The cleaned data was transformed into graph-ready CSV tables such as:

- Plants.csv
- Plant_ailments.csv
- Plant_families.csv
- Plant_locations.csv
- Plant_shops.csv
- Categories.csv

Neo4j loading

The data was loaded into Neo4j using Cypher. Constraints were created to prevent duplication, and the graph was loaded with additional derived properties later in the pipeline.

Validation

Validation queries confirmed that the graph had loaded correctly, with expected node and relationship counts.

<pre>2 // STEP 14: VALIDATE GRAPH 3 // Node counts 4 5 6 MATCH (n) 7 RETURN labels(n)[0] AS label, count(*) AS count 8 ORDER BY count DESC;</pre>	<pre>1 2 // STEP 14B: VALIDATE GRAPH 3 // Relationship counts 4 5 6 MATCH ()-[r]->() 7 RETURN type(r) AS relationship, count(*) AS count 8 ORDER BY count DESC;</pre>																																
<table><tr><th>label</th><th>count</th></tr><tr><td>"Plant"</td><td>51</td></tr><tr><td>"Ailment"</td><td>42</td></tr><tr><td>"Family"</td><td>34</td></tr><tr><td>"WildLocation"</td><td>26</td></tr><tr><td>"Shop"</td><td>22</td></tr><tr><td>"AilmentCategory"</td><td>13</td></tr><tr><td>"_Bloom_Perspective_"</td><td>1</td></tr><tr><td>"_Bloom_Scene_"</td><td>1</td></tr></table>	label	count	"Plant"	51	"Ailment"	42	"Family"	34	"WildLocation"	26	"Shop"	22	"AilmentCategory"	13	"_Bloom_Perspective_"	1	"_Bloom_Scene_"	1	<table><tr><th>relationship</th><th>count</th></tr><tr><td>1 "TREATS"</td><td>110</td></tr><tr><td>2 "AVAILABLE_AT"</td><td>53</td></tr><tr><td>3 "BELONGS_TO"</td><td>51</td></tr><tr><td>4 "FOUND_IN"</td><td>51</td></tr><tr><td>5 "IN_CATEGORY"</td><td>42</td></tr><tr><td>6 "_Bloom_HAS_SCENE_"</td><td>1</td></tr></table>	relationship	count	1 "TREATS"	110	2 "AVAILABLE_AT"	53	3 "BELONGS_TO"	51	4 "FOUND_IN"	51	5 "IN_CATEGORY"	42	6 "_Bloom_HAS_SCENE_"	1
label	count																																
"Plant"	51																																
"Ailment"	42																																
"Family"	34																																
"WildLocation"	26																																
"Shop"	22																																
"AilmentCategory"	13																																
"_Bloom_Perspective_"	1																																
"_Bloom_Scene_"	1																																
relationship	count																																
1 "TREATS"	110																																
2 "AVAILABLE_AT"	53																																
3 "BELONGS_TO"	51																																
4 "FOUND_IN"	51																																
5 "IN_CATEGORY"	42																																
6 "_Bloom_HAS_SCENE_"	1																																

Figure 3: Node and Relationship validation



Exploratory Statistical and Visual Analysis

Descriptive graph analysis and visual query outputs were done using python and Neo4j sandbox.

Most versatile plants

A count of distinct ailments treated by each plant was used as an indicator of medicinal versatility.

The most versatile plants included:

- Garlic (wild and cultivated)
- Chamomile
- Kelp
- Carrageen moss
- Elder flowers
- Nettles
- Dandelion

These plants each treated four distinct ailments in the current graph.

```
1 // =====
2 // STEP 16A: MOST VERSATILE PLANTS
3 // =====
4
5 MATCH (p:Plant)-[:TREATS]->(a:Ailment)
6 RETURN p.name AS plant, count(DISTINCT a) AS ailmentsTreated
7 ORDER BY ailmentsTreated DESC
8 LIMIT 10;
```

Table	RAW
plant	ailmentsTreated
"Garlic (wild and cultivated)"	4
"Chamomile"	4
"Kelp"	4
"Carrageen moss"	4
"Elder flowers"	4
"Nettles"	4
"Dandelion"	4
"Belladonna / Deadly nightshade"	4
"Burdock"	3
"Comfrey / 'Knit bones'"	3

Figure 6: Most versatile plants

Dominant treatment categories

Category analysis showed that the most frequent medicinal themes were:

- Skin/ Wound
- Pain/ Musculoskeletal
- Respiratory
- General Tonic
- Digestive

This indicates that the traditional remedies in the dataset were especially oriented toward practical everyday conditions such as wounds, pain, respiratory complaints, and general strengthening.

category	links
¹ "Other"	23
² "Skin/Wound"	18
³ "Pain/Musculoskeletal"	16
⁴ "Respiratory"	13
⁵ "General Tonic"	8
⁶ "Digestive"	7
⁷ "Emotional Wellbeing"	7
⁸ "Infection/Fever"	5
⁹ "Circulatory"	5
¹⁰ "Toxic/Poison"	3
¹¹ "Women's Health"	2
¹² "Neurological"	2
¹³ "Eye"	1

Figure 7: Treatment categories

Access pattern analysis

A refined access-pattern query showed:

- Retail Fresh = 26 plants
- Retail Dry = 12 plants
- Wild Only = 12 plants
- Mixed (Wild + Retail) = 1 plant

This is a strong result because it reveals that although many remedies are available through modern retail systems, a significant subset remains dependent on wild access. This supports the argument that traditional ecological knowledge still matters even in the modern herbal market context.

neo4j\$ // ===== // RESULT 9: ACCESS PATTERN ANALYSIS /	
Table RAW	
accessPattern	plantCount
¹ "Retail Fresh"	26
² "Retail Dry"	12
³ "Wild Only"	12
⁴ "Mixed Access"	1

Figure 8: Access patterns

Safety analysis

A toxicity-aware filter identified low-toxicity plants with multiple uses, including:

- Carrageen moss
- Chamomile
- Dandelion
- Elder flowers
- Garlic
- Kelp
- Nettles

This provided a practical safety-oriented perspective for later deployment.

					
	safeSuperPlant	numUses	uses		
1	"Carrageen moss"	4	["Arthritis", "Rheumatism", "Bronchitis", "Digestion"]		
2	"Chamomile"	4	["Skin", "Women's Health", "Calming", "Chicken pox"]		
3	"Dandelion"	4	["General Health", "Diuretic", "Remove warts", "Bladder"]		
4	"Elder flowers"	4	["Fever", "Cold", "Pancakes", "Fizzy drink"]		
5	"Garlic (wild and cultivated)"	4	["Circulation", "Bronchitis", "Cold", "Chillblains"]		
6	"Kelp"	4	["General Health", "Arthritis", "Rheumatism", "Digestion"]		
7	"Nettles"	4	["General Health", "Boils", "Circulation", "Spring soup"]		
8	"Burdock"	3	["Boils", "Arthritis", "Rheumatism"]		
9	"Feverfew"	3	["Arthritis", "Rheumatism", "Headache"]		
10	"Onion"	3	["Poison", "Cold", "Stings"]		
11	"Greater plantain"	3	["Stings", "Wounds", "Varicose ulcers"]		

Figure 9: Safety patterns

Knowledge Graph Design and Visualization

The graph was implemented as a Neo4j property graph. The Plant node served as the central entity, linked to ailments, categories, locations, families, and shops. This created the project a strong structure and supported explainable queries.

A case-study visualization of **Chamomile** was especially useful because it clearly demonstrated how a single plant node could connect to:

- Multiple therapeutic uses
- A family
- A shop
- A wild location

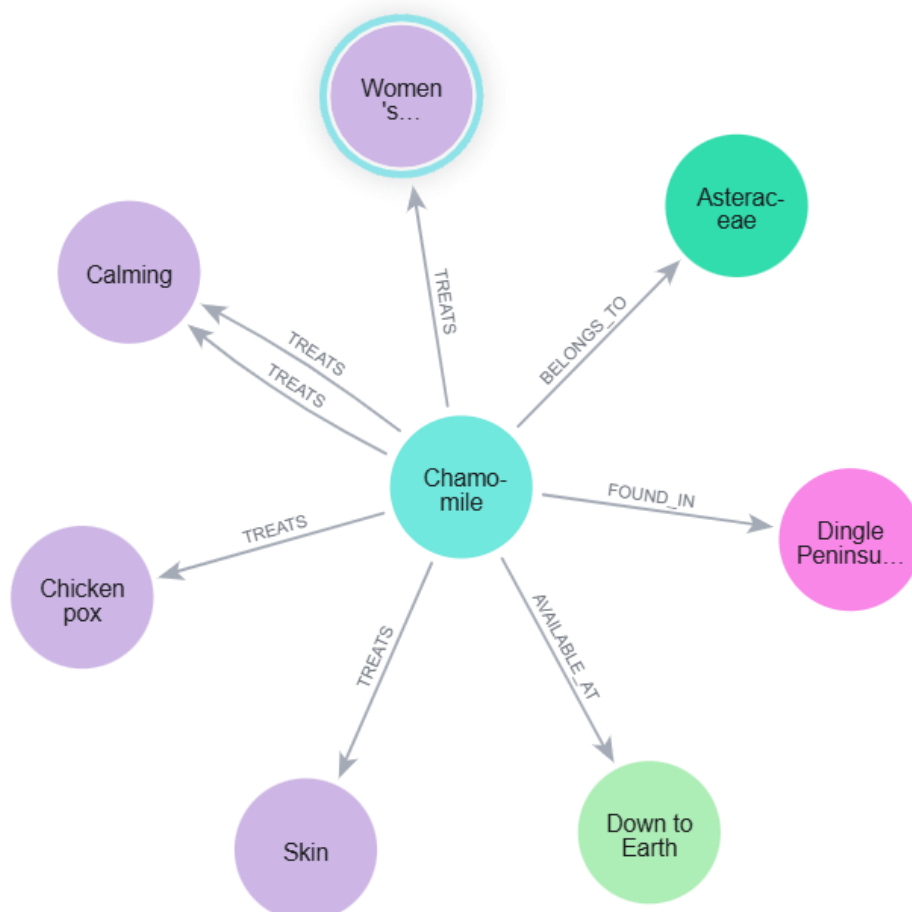


Figure 10: Chamomile node

Modelling: Graph Algorithms and Graph Learning

Graph algorithms and graph-learning use cases were applied through centrality, betweenness, Louvain community detection, node similarity, and embedding-based feature generation.

Degree Centrality

Degree centrality was used to identify highly connected plants in the Plant-Ailment graph. It can be used as a measure of structural importance in the network.

Top plants by degree centrality included:

- Chamomile
- Carrageen moss
- Comfrey/ ‘Knit boes’
- Nettles
- Dandelion
- Elder flowers
- Kelp

These results overlap strongly with the versatility analysis, which supports the idea that highly connected graph nodes also tend to be multi-use medicinal plants.

```
7 WITH gds.util.asNode(nodeId) AS n, score
8 WHERE n:Plant
9 RETURN n.name AS plant, score AS degreeCentrality
10 ORDER BY degreeCentrality DESC
11 LIMIT 10;
```

plant	degreeCentrality
1 "Chamomile"	5.0
2 "Carrageen moss"	5.0
3 "Comfrey / 'Knit bones'"	5.0
4 "Nettles"	5.0
5 "Dandelion"	4.0
6 "Elder flowers"	4.0
7 "Kelp"	4.0
8 "Belladonna / Deadly nightshade"	4.0
9 "Garlic (wild and cultivated)"	4.0
10 "Burdock"	3.0

Figure 31 : Degree centrality

Betweenness Centrality

Betweenness centrality was used as a secondary graph measure to identify bridge plants. It shows how often a plant appears on the shortest paths connecting other nodes. Some outputs were less directly meaningful, which is normal in a sparse bipartite medicinal graph.

Therefore, betweenness was treated as an indicator of structural connectivity rather than as a direct ranking of the most practically used medicinal plants.

```
1 // =====
2 // STEP 19A: BETWEENNESS TABLE
3 // Finds bridge plants
4 // =====
5
6 CALL gds.betweenness.stream('ailmentGraph')
7 YIELD nodeId, score
8 WITH gds.util.asNode(nodeId) AS n, score
9 WHERE n:Plant
10 RETURN n.name AS plant, round(score, 3) AS betweenness
11 ORDER BY betweenness DESC
12 LIMIT 10;
```

	plant	betweenness
1	"Garlic (wild and cultivated)"	547.984
2	"Onion"	522.626
3	"Belladonna / Deadly nightshade"	512.524
4	"Kelp"	473.62
5	"Nettles"	398.085
6	"Peat moss / Turf"	259.833
7	"Elder flowers"	257.201
8	"Chamomile"	223.659
9	"Carrageen moss"	218.978

Figure 42: Betweenness centrality

Louvain Community Detection

Louvain was used to identify natural communities in the graph. It grouped plants and ailments into clusters based on shared connections. These communities aligned with themes such as:

- Respiratory remedies
- Skin and wound remedies
- Digestive remedies
- Pain-related remedies

This was important because it demonstrated that the graph naturally formed meaningful medicinal groups, rather than relying on categories that were assigned manually.

```
2 // STEP 20A: LOUVAIN TABLE
3 // Detect therapeutic clusters
4 // =====
5
6 CALL gds.louvain.stream('ailmentGraph')
7 YIELD nodeId, communityId
8 WITH gds.util.asNode(nodeId) AS n, communityId
9 RETURN labels(n)[0] AS nodeType,
10        coalesce(n.name, n.irishName) AS entity,
11        communityId
12 ORDER BY communityId, nodeType, entity;
```

Table RAW

nodeType	entity
"Ailment"	"Flowers eaten or in a drink for a cooling effect"
"Plant"	"Bee borage"
"Ailment"	"Calming"
"Ailment"	"Chicken pox"
"Ailment"	"Digestion"
"Ailment"	"Skin"
"Plant"	"Chamomile"
"Plant"	"Flax"
"Plant"	"Lavender oil"

Figure 53: Louvain table

Node Similarity

Node similarity was used as the main graph AI/ recommendation method. It identified structurally similar plants based on shared graph relationships.

Examples included:

- Greater plantain ↔ Lavender oil
- Dandelion ↔ Bogbean
- Spearmint ↔ Peppermint

The Peppermint-Spearmint pair is particularly interpretable and clearly defined as a recommendation example.

```
12 p2.name AS plant2,  
13 round(similarity, 3) AS similarity  
14 ORDER BY similarity DESC  
15 LIMIT 15;
```

plant1	plant2	similarity
1 "Greater plantain"	"Lavender oil"	1.0
2 "Lavender oil"	"Greater plantain"	1.0
3 "Dandelion"	"Bogbean"	0.833
4 "Bogbean"	"Dandelion"	0.833
5 "Furze / Gorse"	"Dandelion"	0.8
6 "Dandelion"	"Furze / Gorse"	0.8
7 "House leek"	"Furze / Gorse"	0.75
8 "Furze / Gorse"	"House leek"	0.75
9 "Spearmint"	"Peppermint"	0.75
10 "Peppermint"	"Spearmint"	0.75

Figure 14: Node similarity

KNN on graph embeddings

The project also analysis a KNN-based recommendation step using graph embeddings. This provided a more advanced graph-ML style extension of basic similarity.

```
15     gds.util.asNode(node2) AS p2,
16     similarity
17 WHERE p1:Plant AND p2:Plant
18 RETURN p1.name AS plant1,
19        p2.name AS plant2,
20        round(similarity, 3) AS similarity
21 ORDER BY similarity DESC
22 LIMIT 15;
```

plant1	plant2	similarity
6 "Aloe vera"	"Hart's tongue fern"	1.0
7 "Furze / Gorse"	"House leek"	1.0
8 "Peas"	"Senna"	1.0
9 "Senna"	"Peas"	1.0
10 "Silverweed"	"Charlock"	1.0
11 "Hart's tongue fern"	"Aloe vera"	1.0
12 "Charlock"	"Silverweed"	1.0
13 "Spearment"	"Peppermint"	0.991
14 "Peppermint"	"Spearment"	0.991
15 "Meadowsweet"	"Comfrey / 'Knit bones'"	0.981

Figure 15: KNN similarity

Machine Learning Extension

To take the project further while keeping the graph-theory core intact, graph-derived features were exported into a plant-level machine learning table. And it used graph outputs as features.

plant_id	plant_name	irish_na	family	famine_	toxicity_	ailment_	category_	shop_cc	location	degree_	between	commur	access_	safe_multi_use
1 "aloe_vera_l_bu rm_f"	"Aloe vera"	"N/A"	"Aspho delace ae"	false	"Low"	1	1	1	1	1.0	0.0	85	"Retai l Fres h"	false
2 "arnica_montana _l"	"Arnica"	"N/A"	"Aster aceae"	false	"Low"	2	1	1	1	2.0	148.86 008403 361348	90	"Retai l Dry"	false
3 "borago_officin alis_l"	"Bee borage"	"Borra irte g orm"	"Borag inacea e"	false	"Low"	1	1	2	1	1.0	0.0	51	"Retai l Fres h"	false
4 "atropa_bellado nna_l"	"Belladonna / Deadly nig htshade"	"Miotó g buid e"	"Solan aceae"	false	"High"	4	4	1	1	4.0	512.52 362711 88318	83	"Retai l Fres h"	false
5 "rubus_spp"	"Blackberry"	"Sceac h / Dr is"	"Rosac eae"	false	"Low"	1	1	2	1	1.0	0.0	56	"Mixed Acces s"	false
6 "menyanthes_tri foliata_l"	"Bogbean"	"Bachr ân"	"Menya nthace ae"	false	"Low"	2	2	1	1	3.0	71.0	73	"Retai l Dry"	false
7 "arctium_lappa_ l"	"Burdock"	"Meaca n dog a"	"Aster aceae"	false	"Low"	3	2	1	1	3.0	100.20 738355 212038	60	"Retai l Fres h"	true
8 "brassica_olera cea_l"	"Cabbage"	"Cabái ete"	"Brass icaceae"	false	"Low"	2	2	1	1	2.0	94.963 595777	62	"Retai l Fres h"	false

Figure 16: Graph outputs

Feature engineering

Each plant was represented by graph-derived variables such as:

- Ailment count
- Category count
- Shop count
- Location count
- Degree centrality
- Betweenness
- Community ID
- Toxicity class
- Access pattern
- Safe multi-use flag

This was an important methodological contribution because it demonstrated how graph analytics can feed into ML tasks.

```
# STEP 27: Load features extracted from Neo4j
```

```
# -----
uploaded = files.upload()
```

```
feature_file = "plant_graph_features_V2.csv"
features_df = pd.read_csv(feature_file)
```

```
print(features_df.head())
print(features_df.shape)
```

Choose Files | No file chosen | Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving plant_graph_features_V2.csv to plant_graph_features_V2 (1).csv

	plant_id	plant_name	irish_name
0	aloe_vera_l_burm_f	Aloe vera	NaN
1	arnica_montana_l	Arnica	NaN
2	borago_officinalis_l	Bee borage	Borraitte gorm
3	atropa_belladonna_l	Belladonna / Deadly nightshade	Miotóg buide
4	rubus_spp	Blackberry	Sceach / Dris

	family	famine_food	toxicity_class	ailment_count	category_count
0	Asphodelaceae	False	Low	1	1
1	Asteraceae	False	Low	2	1
2	Boraginaceae	False	Low	1	1
3	Solanaceae	False	High	4	4
4	Rosaceae	False	Low	1	1

	shop_count	location_count	degree Centrality	betweenness	community_id
0	1	1	1.0	0.000000	85
1	1	1	2.0	148.860084	90
2	2	1	1.0	0.000000	51
3	1	1	4.0	512.523627	83
4	2	1	1.0	0.000000	56

	access_pattern	safe_multi_use
0	Retail Fresh	False
1	Retail Dry	False
2	Retail Fresh	False
3	Retail Fresh	False

Figure 17: Graph output summary

K-means clustering

K-means clustering was applied to the graph-derived feature table.

The clustering analysis produced:

- 4 plant clusters
- Silhouette score = 0.339

This indicates moderate level of separation between groups. Clusters were meaningful and easy to interpret.

- One cluster represented highly connected, versatile, generally safe plants
- Another represented herbs with moderate range of uses.
- The third group with more specialized herbs with fewer connections.
- Another represented more visible through retail access but structurally simpler herbs

The clustering therefore provided an additional unsupervised view of medicinal plant structure, confirming the Louvain graph community results.

```
# STEP 30: K-Means clustering
# -----
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
ml_df["k_cluster"] = kmeans.fit_predict(X_cluster_scaled)

print(ml_df[["plant_name", "k_cluster"]].head(15))
print("Silhouette score:", silhouette_score(X_cluster_scaled, ml_df["k_cluster"]))
```

```
      plant_name  k_cluster
0      Aloe vera         2
1       Arnica          2
2  Bee borage         3
3  Belladonna / Deadly nightshade  1
4     Blackberry         3
5      Bogbean          0
6     Burdock          1
7     Cabbage          0
8  Carrageen moss         1
9    Chamomile          1
10   Charlock          2
11     Cloves          2
12  Comfrey / 'Knit bones'         1
13   Dandelion          1
14      Dock          2
Silhouette score: 0.33930254675259347
```

```
# STEP 31: Cluster Interpretation
# -----
cluster_summary = ml_df.groupby("k_cluster")[cluster_features].mean().round(2)
print(cluster_summary)

cluster_members = ml_df.groupby("k_cluster")["plant_name"].apply(list)
print(cluster_members)
```

```
      ailment_count  category_count  shop_count  location_count  \
k_cluster
0             1.73             1.73           1.0             1.0
1             3.62             2.92           1.0             1.0
2             1.19             1.10           1.0             1.0
3             1.00             1.00           2.0             1.0

      degree centrality  betweenness  community_id  toxicity_numeric  \
k_cluster
0             2.00         69.08         58.33             0.00
1             4.00        291.57         69.85             0.23
2             1.24         21.92         84.29             0.24
3             1.00          0.00         53.50             0.00

      famine_food  safe_multi_use
k_cluster
0             0.00             0.00
1             0.08             0.85
2             0.14             0.00
3             0.00             0.00

k_cluster
0  [Bogbean, Cabbage, Dulce, Flax, Ivy, Lavender ...
1  [Belladonna / Deadly nightshade, Burdock, Carr...
2  [Aloe vera, Arnica, Charlock, Cloves, Dock, Ea...
3  [Bee borage, Blackberry]
Name: plant_name, dtype: object
```

Figure 18: K-means clustering results

Lasso regression

Lasso regression was used as an interpretable predictive extension, with **ailment count** as the target variable. The model achieved:

- $R^2 = 0.882$
- $RMSE = 0.459$

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler_reg = StandardScaler()
X_train_scaled = scaler_reg.fit_transform(X_train)
X_test_scaled = scaler_reg.transform(X_test)

lasso = Lasso(alpha=0.1, random_state=42)
lasso.fit(X_train_scaled, y_train)

y_pred = lasso.predict(X_test_scaled)

print("R2:", r2_score(y_test, y_pred))

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

```
R2: 0.8819489665454631
RMSE: 0.4592984881346574
```

```
# STEP 34: LASSO Feature Importance
# -----
coef_df = pd.DataFrame({
    "feature": X.columns,
    "coefficient": lasso.coef_
}).sort_values("coefficient", key=abs, ascending=False)

print(coef_df.head(15))

plt.figure(figsize=(10, 6))
plt.bar(coef_df.head(15)["feature"], coef_df.head(15)["coefficient"])
plt.xticks(rotation=75, ha="right")
plt.title("Top Lasso Coefficients for Predicting Plant Versatility")
plt.tight_layout()
plt.show()
```

	feature	coefficient
4	degree centrality	0.470321
7	safe_multi_use	0.180813
1	category_count	0.115320
5	betweenness	0.083953
3	location_count	0.000000
0	famine_food	-0.000000
2	shop_count	-0.000000
6	community_id	-0.000000
8	toxicity_numeric	-0.000000
9	family_Araliaceae	0.000000
10	family_Aspodelaceae	-0.000000
11	family_Aspleniaceae	-0.000000
12	family_Asteraceae	0.000000
13	family_Boraginaceae	-0.000000
14	family_Brassicaceae	0.000000

Figure 19: Lasso feature importance

This suggests that graph-derived features explain a large proportion of variation in medicinal versatility within the dataset.

The strongest coefficients were:

- Degree centrality
- Safe multi-use
- Category count
- Betweenness

This is one of the strongest findings in the extended project, because it shows that graph-based measures are useful not only for describing the structure of the network, for predicting how versatile a medical plant is.

Interpretation

The ML extension supports two important conclusions:

1. Graph-derived features can be used to meaningfully cluster medicinal plants into groups with similar structural and practical characteristics.
2. Graph-derived measures such as degree centrality, category count, safe multi use status, and betweenness can help explain variation in plant versatility.

This strengthens the project by adding graph-theory core into machine learning setting while keeping graph structure as the central of the analysis.

Evaluation

Evaluation of graph algorithms

Graph algorithm outputs were evaluated primarily through:

- Interpretability
- Domain relevance
- Structural coherence

Degree centrality was considered strong because the most connected plants were also plausibility of multi-use medicinal herbs. Louvain community detection was also meaningful because they aligned with recognizable medicinal themes such as respiratory, digestive, skin and pain related remedies. Node similarity was considered strong where plant pairs were clearly interpretable, such as Peppermint and Spearmint. Betweenness centrality was used as structural bridge of information only.

Evaluation of graph ML

For node similarity and graph-based recommendation logic, evaluation relied on:

- Plausibility of plant pairs
- Medicinal coherence
- Usefulness for alternative recommendation

Evaluation of K-means

The silhouette score of 0.339 indicates moderate cluster quality. This suggests that the clusters are meaningful but not perfectly separated, which is realistic for a small, heterogeneous ethnobotanical graph-derived dataset. But the cluster outputs were still useful as they create additional unsupervised view of plant and network supporting Louvain community detection results.

Evaluation of Lasso

The Lasso model achieved strong explanatory performance. However, because the dataset is relatively small, the results should be seen as early insight into understanding patterns, not as final or fully reliable predictions.

Evaluation of Logistic Regression

Logistic regression achieved perfect classification performance on the test set, with 100% accuracy, precision, recall and F1-score. This clarifies that model was able to clarify separate multi use plants from other plants in the available data. But since datasets are small results should be seen as early insight into understanding patterns.

```
# STEP 35: LOGISTIC Regression - safe multi use
# -----
target_cls = "safe_multi_use"

predictor_features_cls = [
    col for col in ml_df.columns
    if col not in [
        "plant_id", "plant_name", "irish_name",
        "toxicity_class", target_cls
    ]
]

X_cls = ml_df[predictor_features_cls].fillna(0)
y_cls = ml_df[target_cls]

X_train, X_test, y_train, y_test = train_test_split(
    X_cls, y_cls, test_size=0.25, random_state=42, stratify=y_cls
)

scaler_cls = StandardScaler()
X_train_scaled = scaler_cls.fit_transform(X_train)
X_test_scaled = scaler_cls.transform(X_test)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred_cls = log_reg.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_cls))
print(classification_report(y_test, y_pred_cls))
```

```
Accuracy: 1.00
      precision    recall  f1-score   support

     0         1.00      1.00      1.00        10
     1         1.00      1.00      1.00         3

   accuracy          1.00
  macro avg          1.00
 weighted avg          1.00
```

Figure 20: Logistic regression results

Limitations

- Small dataset size
- Some manually normalized ailment labels
- Category “Other” still captures some heterogeneous uses that are not yet fully refined.
- Recommendation quality is based on structural similarity rather than clinical validation
- The ML extension should be considered as an early-stage analysis due to limited dataset scale
- Limited resources and prior research and systems to confirm validity of the results.

Deployment

This project supports a realistic deployment concept in the capacity of remedy assistant build on top of the Neo4j knowledge graph.

Condition-to-remedy assistant

A user can enter a condition such as:

- Cold
- Respiratory
- Skin/ Wound

The graph returns:

- Matching herbs
- Toxicity level
- Wild locations
- Retail shops in Dublin

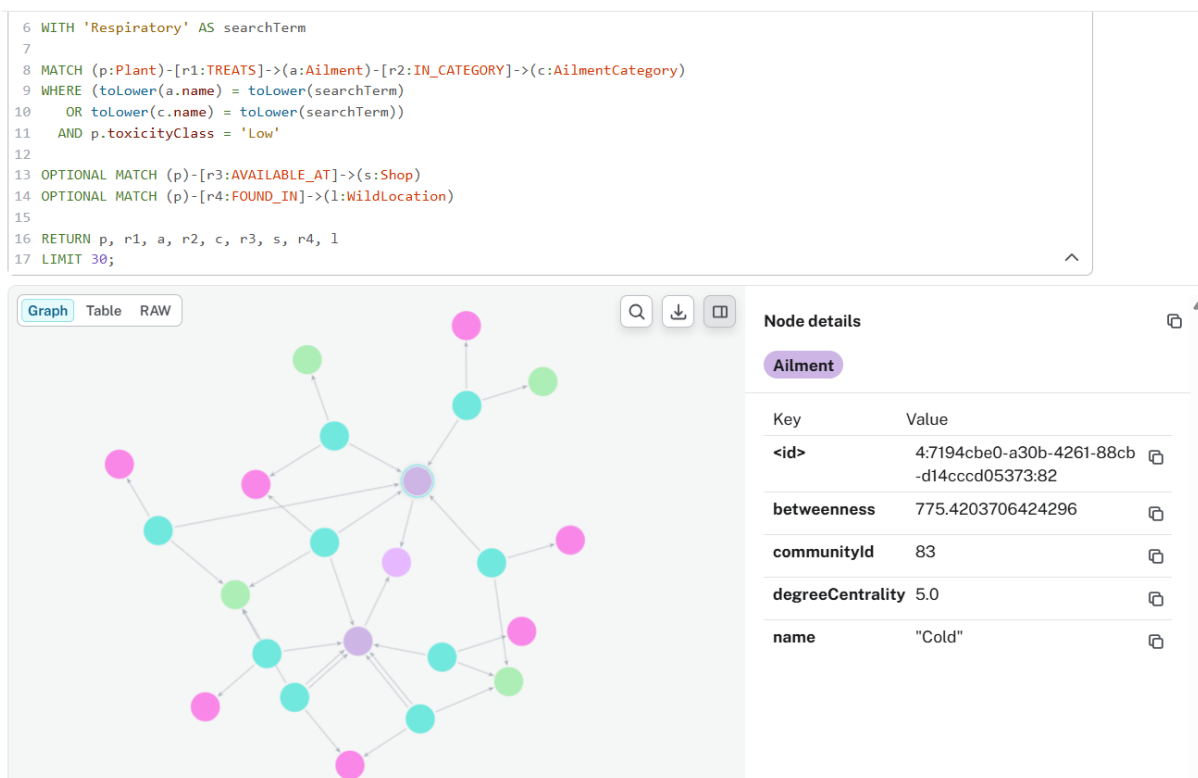


Figure 21: Remedy assistant model

Safety-first version

A safety-first filter restricts results to low-toxicity herbs, making the query more practical for a public-facing recommendation system.

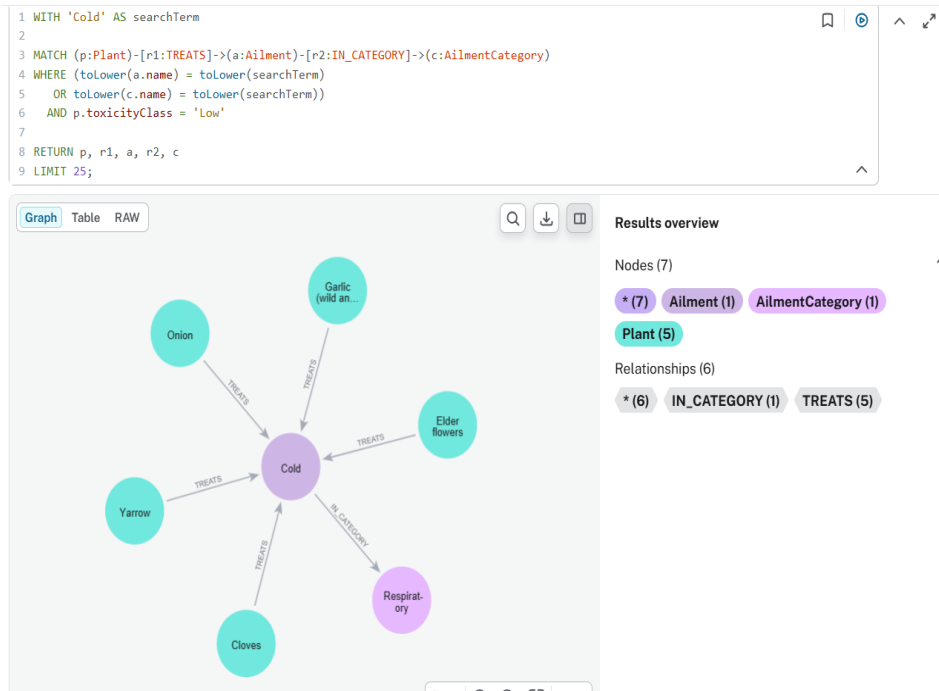


Figure 22: Remedy assistant safety model

Explainability

Because the system is graph-based, each recommendation can be traced through visible relationship of the graph, allowing system to show.

- Why was the herb returned
- Which ailment it treats
- Which category it belongs to
- Whether it is safe
- Where can it be found or bought

Future deployment

Possible next steps include:

- A platform or mobile app contains chatbot interface over Neo4j
- Image-based plant recognition
- Integration with a hosted Neo4j Sandbox/ Aura/ Bloom setup
- Provenance links from graph facts back to source references
- Ability to get confirmation from medical associates for result verification through the suggested system
- Explore large range of traditional medicine plants in Ireland and across the Europe and distribute their use of knowledge across the society
- Establish a location where people can explore medicines and get knowledge about them and buy all the medicines and plants they need for day-to-day life remedies. improving usability, scalability and trustworthiness of the system.

Conclusion

This project successfully demonstrates how traditional Irish medicinal knowledge can be transformed into an intelligent graph-based system. By modelling plants, ailments, categories, access points, and structural features in Neo4j, the project made the domain queryable, explainable, and suitable for advanced analytics.

The graph-theory core of the project was preserved through:

- Graph schema design
- Centrality analysis
- Community detection
- Similarity analysis
- Access and safety reasoning

On top of this, the project was extended with:

- Graph-derived feature engineering
- K-means clustering
- Lasso regression
- Deployment-oriented condition search

The strongest overall conclusion is that **graph structure contains meaningful medicinal knowledge**. Highly connected plants were also highly versatile, graph similarity can support recommendation, and graph-derived variables can explain important outcomes such as plant versatility.

Overall, this project demonstrates how graph-based AI can transform traditional knowledge into an intelligent, explainable, and practical system.

References

Dolan, J.M., 2007. Ochtriril's legacy: Irish women's knowledge of medicinal plants. *Harvard Papers in botany*, 12(2), pp.369-386.

<https://neo4j.com/docs/>

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html

AI Declaration: AI assistance was used to improve ideas and cross check correctness of the selected models.